



Mini-cours : Prendre une photo sous Android

Deux méthodes : Intent vs CameraX

1. Introduction

Prendre une photo dans une application Android peut se faire de deux manières principales :

1. **Via un Intent** : Déléguer la prise de photo à une application externe.
2. **Via CameraX** : Contrôler entièrement la caméra dans son application.

Chaque méthode a ses avantages et inconvénients. Nous allons les explorer en détail avec des exemples de code et des bonnes pratiques.

2. Méthode 1 : Utiliser un Intent

Principe

Utiliser un **Intent** pour lancer l'application photo par défaut de l'appareil. Cette méthode est simple mais offre peu de contrôle sur l'interface et les paramètres de la caméra.

Étapes Clés

a. Déclarer les permissions

Dans le **AndroidManifest.xml** :

```
<uses-permission android:name="android.permission.CAMERA" />
```

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

Note : Depuis Android 10 (API 29), l'accès au stockage externe est restreint. Préférez sauvegarder les photos dans le dossier spécifique de votre application.

b. Lancer l'Intent pour prendre une photo

Dans ton Activity ou Fragment :

```
val REQUEST_IMAGE_CAPTURE = 1
val takePictureIntent = Intent(MediaStore.ACTION_IMAGE_CAPTURE)
startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE)
```

c. Récupérer la photo

Gérer le résultat dans `onActivityResult` :

```
override fun onActivityResult(requestCode: Int, resultCode: Int, data:
Intent?) {
    super.onActivityResult(requestCode, resultCode, data)
    if (requestCode == REQUEST_IMAGE_CAPTURE && resultCode == RESULT_OK) {
        val imageBitmap = data?.extras?.get("data") as Bitmap
        // Afficher ou sauvegarder l'image
        saveImageToGallery(imageBitmap)
    }
}
```

d. Sauvegarder l'image

Pour sauvegarder dans la galerie :

```
private fun saveImageToAppDirectory(bitmap: Bitmap) {

    // Créer un dossier pour votre application, s'il n'existe pas déjà
    val appSpecificExternalDir = File(getExternalFilesDir(null),
"MyAppPhotos")

    if (!appSpecificExternalDir.exists()) {
        appSpecificExternalDir.mkdirs()
    }

    // Créer un fichier pour l'image
    val fileName = "photo_${System.currentTimeMillis()}.jpg"
    val file = File(appSpecificExternalDir, fileName)
```

```
try {  
  
    // Écrire le Bitmap dans le fichier  
  
    FileOutputStream(file).use { outputStream ->  
  
        bitmap.compress(Bitmap.CompressFormat.JPEG, 100, outputStream)  
  
        outputStream.flush()  
  
    }  
  
    // Afficher un message de confirmation  
  
    Toast.makeText(this, "Photo sauvegardée dans le dossier de  
l'application", Toast.LENGTH_SHORT).show()  
  
    } catch (e: Exception) {  
  
        Toast.makeText(this, "Erreur lors de la sauvegarde de la photo:  
${e.message}", Toast.LENGTH_SHORT).show()  
  
        e.printStackTrace()  
  
    }  
}
```

Avantages et Inconvénients

Simple et rapide à implémenter
Peu de contrôle sur l'interface
Pas besoin de gérer la preview
Dépend de l'application photo installée
Code minimal
Moins de personnalisation possible

3. Méthode 2 : Utiliser CameraX

Principe

CameraX est une bibliothèque moderne qui offre un contrôle total sur la caméra, y compris la prévisualisation (preview), la capture, et la gestion des images. C'est la méthode recommandée pour des applications qui nécessitent une expérience utilisateur personnalisée.

Étapes Clés

a. Ajouter les dépendances

Importer les modules :

```
com.androidx.camera:camera-core:1.3.0
com.androidx.camera:camera-camera2:1.3.0
com.androidx.camera:camera-lifecycle:1.3.0
com.androidx.camera:camera-view:1.3.0
```

b. Déclarer les permissions

Dans le `AndroidManifest.xml` :

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-feature android:name="android.hardware.camera" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

c. Initialiser CameraX et configurer la prévisualisation

Dans ton Activity :

1. Vérifier et demander les permissions :

```
private val REQUIRED_PERMISSIONS = arrayOf(Manifest.permission.CAMERA)
private val REQUEST_CODE_PERMISSIONS = 10

private fun allPermissionsGranted() = REQUIRED_PERMISSIONS.all {
    ContextCompat.checkSelfPermission(baseContext, it) ==
    PackageManager.PERMISSION_GRANTED
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    if (allPermissionsGranted()) {
        startCamera()
    } else {
        ActivityCompat.requestPermissions(this, REQUIRED_PERMISSIONS,
        REQUEST_CODE_PERMISSIONS)
    }

    1. }

```

2. Configurer CameraX :

```
private fun startCamera() {
    val cameraProviderFuture = ProcessCameraProvider.getInstance(this)
    cameraProviderFuture.addListener({
        val cameraProvider = cameraProviderFuture.get()
        val preview = Preview.Builder().build().also {
            it.setSurfaceProvider(binding.previewView.surfaceProvider)
        }
        val imageCapture = ImageCapture.Builder().build()
        val cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA
        try {

```

```

        cameraProvider.unbindAll()
        cameraProvider.bindToLifecycle(
            this, cameraSelector, preview, imageCapture)
    } catch (exc: Exception) {
        Log.e("CameraX", "Use case binding failed", exc)
    }
}, ContextCompat.getMainExecutor(this))

```

1. }

d. Prendre une photo

Ajouter un bouton pour capturer une photo :

```

binding.captureButton.setOnClickListener {
    takePhoto()
}

private fun takePhoto() {
    val imageCapture = imageCapture ?: return
    val photoFile = File(externalMediaDirs.first(),
        "${System.currentTimeMillis()}.jpg")
    val outputOptions =
        ImageCapture.OutputFileOptions.Builder(photoFile).build()
    imageCapture.takePicture(
        outputOptions,
        ContextCompat.getMainExecutor(this),
        object : ImageCapture.OnImageSavedCallback {
            override fun onImageSaved(outputFileResults:
                ImageCapture.OutputFileResults) {
                Toast.makeText(this@MainActivity, "Photo sauvegardée:
                ${photoFile.absolutePath}", Toast.LENGTH_SHORT).show()
            }
            override fun onError(exception: ImageCaptureException) {
                Log.e("CameraX", "Erreur lors de la capture:
                ${exception.message}", exception)
            }
        })
}

```

e. Basculer entre les caméras

Ajouter un bouton pour changer de caméra :

```

private var cam = "BACK" // ou "FRONT"

binding.switchCameraButton.setOnClickListener {
    cam = if (cam == "BACK") "FRONT" else "BACK"
    startCamera()
}

```

Avantages et Inconvénients

Contrôle total sur l'interface
Plus complexe à implémenter
Personnalisation complète
Nécessite la gestion du cycle de vie
Compatible avec les dernières versions d'Android
Code plus long et plus complexe

4. Comparaison des Méthodes

Contrôle

Faible
Total

Personnalisation

Limitée
Complète

Complexité

Simple
Modérée à complexe

Compatibilité

Dépend de l'app photo
Indépendant

Gestion de la preview

Non
Oui

Sauvegarde des photos

Simple
Flexible

5. Quel méthode choisir ?

- **Utilise un Intent si :**
 - Tu veux une solution rapide et simple.
 - Tu n'as pas besoin de personnaliser l'interface de la caméra.
 - **Utilise CameraX si :**
 - Tu veux une expérience utilisateur personnalisée.
 - Tu as besoin de contrôler la prévisualisation ou d'autres paramètres de la caméra.
 - Tu veux être indépendant de l'application photo installée sur l'appareil.
-

Conclusion : Les deux méthodes ont leur utilité selon les besoins de votre application. L'important est de bien comprendre les avantages et inconvénients de chacune pour faire le meilleur choix.

6. Ressources et Documentation

- [Documentation CameraX](#)
- [Guide des permissions Android](#)
- [Exemple complet avec CameraX \(GitHub\)](#)